DSA STUDY BLUEPRINT SERIES

49. OOPs Tutorial in One Shot

Classes, Access Modifiers, and Dynamic Polymorphism

Section 07 • C++ Core Data Structures & Algorithms

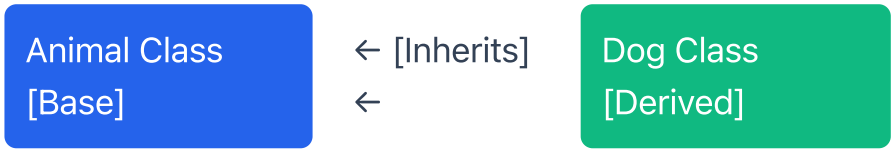
Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Encapsulation:** Hiding attributes behind private access barriers.
- **Polymorphism:** Method Overriding using virtual functions.

Memory & Execution Blueprint



C++ Implementation Reference

Standard code layout and syntax

```
class Animal {
public:
    virtual void sound() { cout << "Animal sound" << endl; }
};
class Dog : public Animal {
public:
    void sound() override { cout << "Woof" << endl; }
};
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Dynamic Dispatch	O(1) (Via V-table pointers)
Object Allocation	O(1)

Key Practices & Gotchas

- **Important:** Use virtual destructors in base classes to prevent memory leaks when deleting derived objects.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace Dynamic Dispatch method lookups in derived objects:

Runtime Object type	Function call	V-Table pointer resolution	Resolved function entry
Dog [Derived]	`animalPtr->sound()`	Checks Dog class Virtual Table pointer	Executes `Dog::sound()` (Prints "Woof!")

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **V-Tables:** Dynamic polymorphism lookup tables map virtual function names to implementation entries at runtime.
- **Virtual Destructors:** Enforces full cleanup calls for derived class members during base pointer deletions.
- **Related LeetCode Problems:** #146 LRU Cache (design tasks), #355 Design Twitter.